



AI-POWERED TELECOM CHURN PREDICTION ASSISTANT

By: Rana Kamal



FEBRUARY 25, 2026

Content

1. Project Overview
2. Development Environment
3. System Architecture
4. Machine Learning Development
5. Backend Implementation
6. Normalization & Semantic Mapping Layer
7. LLM Integration Strategy
8. Streamlit Frontend
9. Risk Analysis & Mitigation
10. Business Insights
11. Technical Strengths
12. How to Use AI Churn Chatbot
13. Future Improvements
14. Conclusion

Chapter 1

Project Overview

This project delivers an end-to-end AI-powered churn prediction system for telecom businesses.

The solution combines:

- Machine Learning (Churn Classification)
- Large Language Model (Natural Language Interface)
- FastAPI Backend
- Streamlit Frontend
- Robust Normalization & Validation Layer

The system enables marketing teams to describe customers in natural language and receive:

- Churn prediction (0 or 1)
- Churn probability
- Risk level (HIGH / LOW)
- Business explanation
- Structured extracted features

The architecture transforms a technical ML model into a usable AI assistant.

Chapter 2

Development Environment

A dedicated Python virtual environment was created specifically for this project to:

- Avoid dependency conflicts
- Ensure reproducibility
- Maintain clean deployment configuration

Core Libraries Used

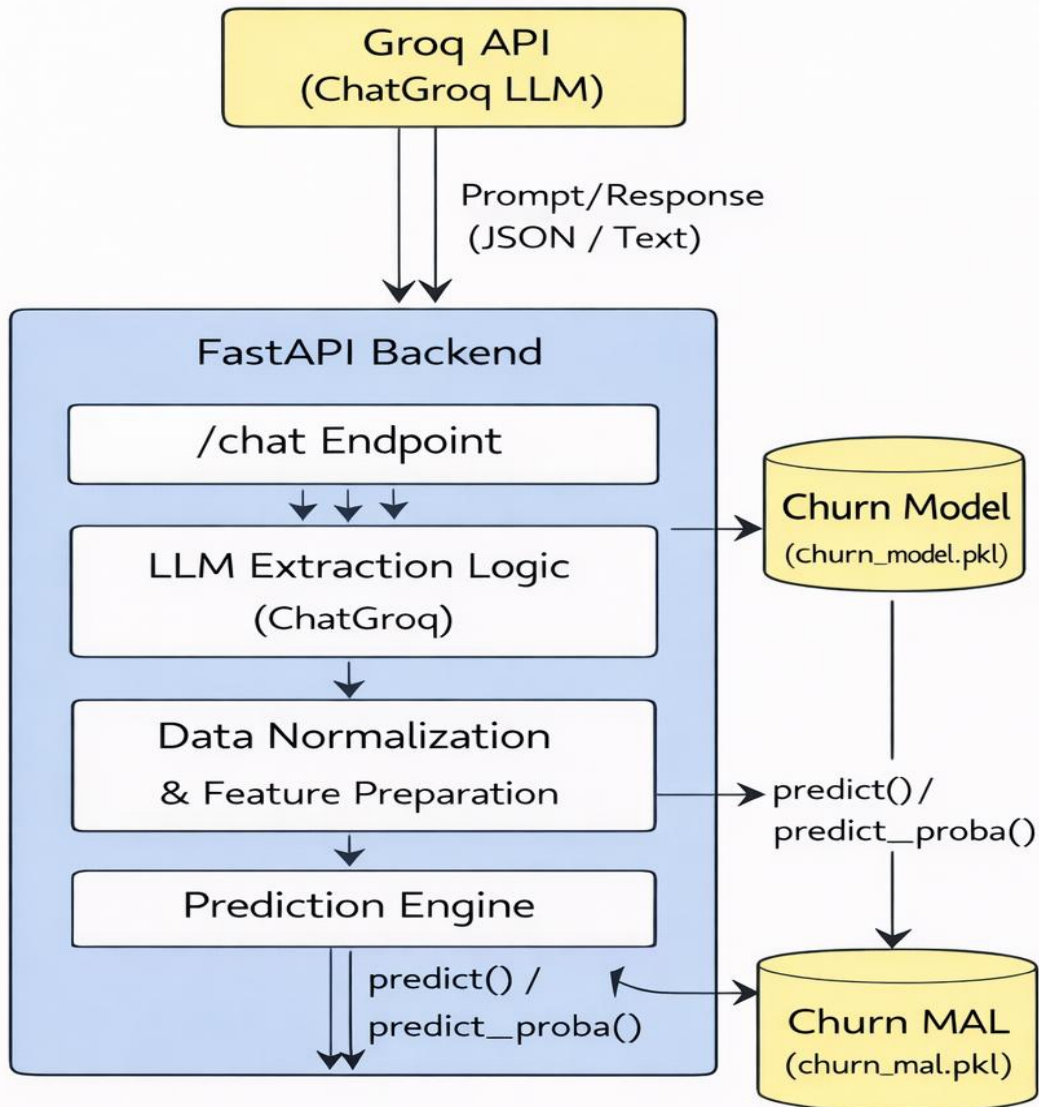
- Python 3.10.19
- Scikit-learn
- Pandas
- NumPy
- FastAPI
- Streamlit
- Joblib
- LangChain
- Groq LLM API

This setup ensures modular and production-ready deployment.

Chapter 3

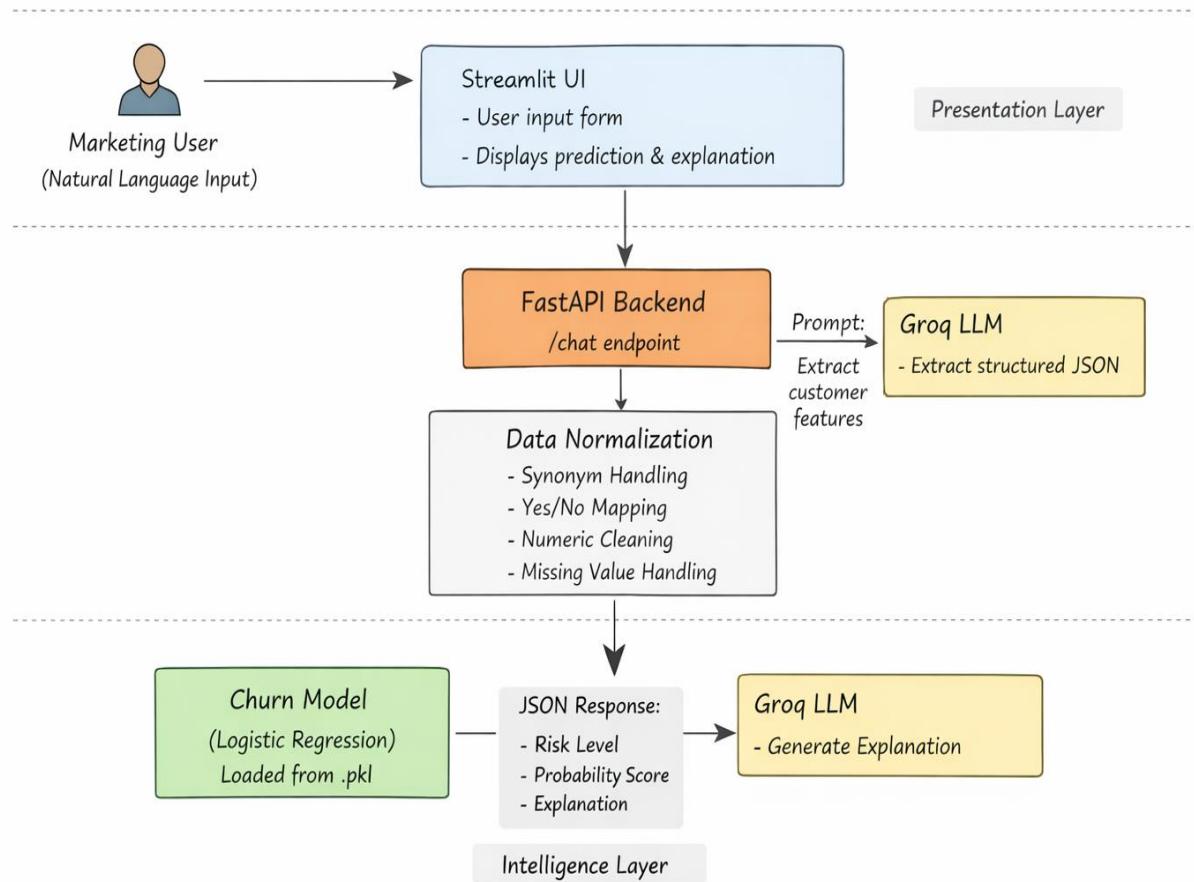
System Architecture

Diagram 2: UML Component



AI-Powered Churn Prediction System Architecture

Hybrid ML + LLM Decision Support System



The system follows a layered architecture:

presentation Layer:

- Streamlit Web App
- Collects Natural Language input
- Displays prediction results

Application Layer:

- FastAPI backend
- /chat Endpoint
- Request Validation
- LLM orchestration

- ML inference
- Error handling

Intelligence Layer:

- Machine Learning model (churn_model.pkl)
- LLaMA 3 (via Groq inference API)

Model Storage Layer:

- Trained ML pipeline saved with preprocessing included

The architecture is modular and model-agnostic.

Chapter 4

Machine Learning Development (AI.ipynb)

Dataset:

- Telco Customer Churn dataset
- 7043 records
- 21 features
- Target: Churn

Preprocessing Strategy

A Scikit-learn Pipeline was built including:

Numerical Features:

- StandardScaler

Categorical Features:

- OneHotEncoder(handle_unknown="ignore")

This ensures safe inference and prevents unseen category errors.

Train/Test Split

- 80% training
- 20% testing
- Stratified by target

Stratification ensures class balance.

Models Trained

Three models were evaluated:

1. Logistic Regression
2. Random Forest (200 estimators)
3. XGBoost (200 estimators)

Evaluation Metrics

Models evaluated using:

- Accuracy
- ROC_AUC (primary metric)
- Precision, Recall, F1-score

ROC_AUC was selected as main metric for churn ranking.

Class Imbalance Handling

Logistic Regression retrained using:

```
class_weight="balanced"
```

Improves recall for churn class.

Model Selection

Best model selected automatically based on highest ROC_AUC.

The entire pipeline (preprocessing + model) was saved:

```
churn_model.pkl
```

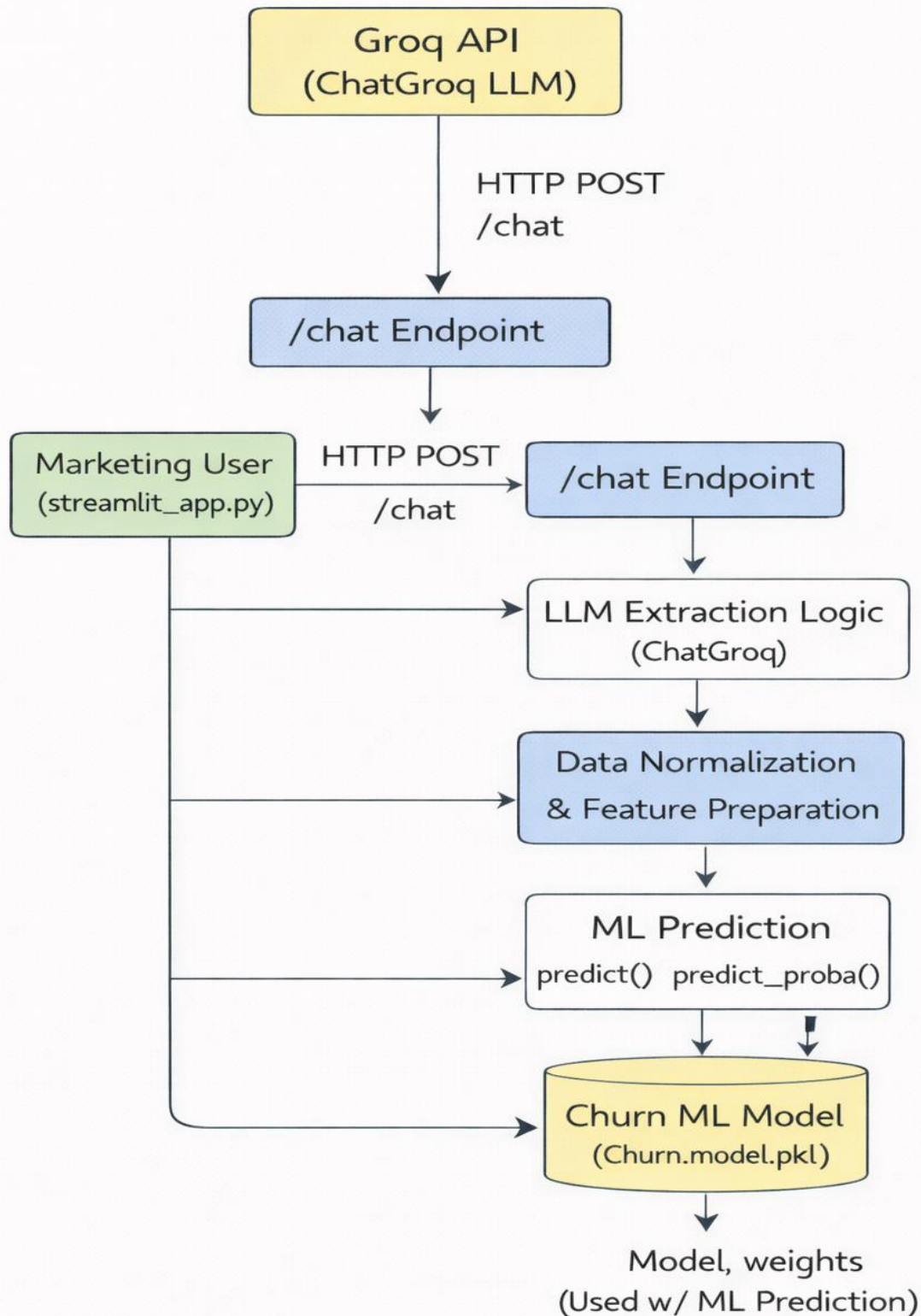
This ensures consistent preprocessing during deployment.

Chapter 5

Backend Implementation

(FastAPI) Endpoint

Diagram 3: Data Flow Diagram



Request Handling Flow

1. Validate request using Pydantic
2. Send extraction prompt to LLM
3. Parse structured JSON safely
4. Apply normalization layer
5. Align features using `model.feature_names_in_`
6. Convert to DataFrame
7. Predict churn
8. Generate explanation via LLM
9. Return structured JSON response

Defensive Programming Implemented

- Safe JSON extraction using `{}` detection
- Multiple try/except blocks
- Numeric type casting fallback
- Missing feature auto-fill
- Schema alignment enforcement
- Probability-based risk classification

Chapter 6

Normalization & Semantic Mapping Layer

A dedicated normalization layer was implemented inside the backend.

Purpose:

To convert natural language variability into model-compatible categories.

Examples:

User Input >> Normalized

"24 months" >> "Two year"

"12 months" >> "One year"

"lady" >> "Female"

"fiber optics" >> "Fiber optic"

"card payment" >> "Credit card (automatic)"

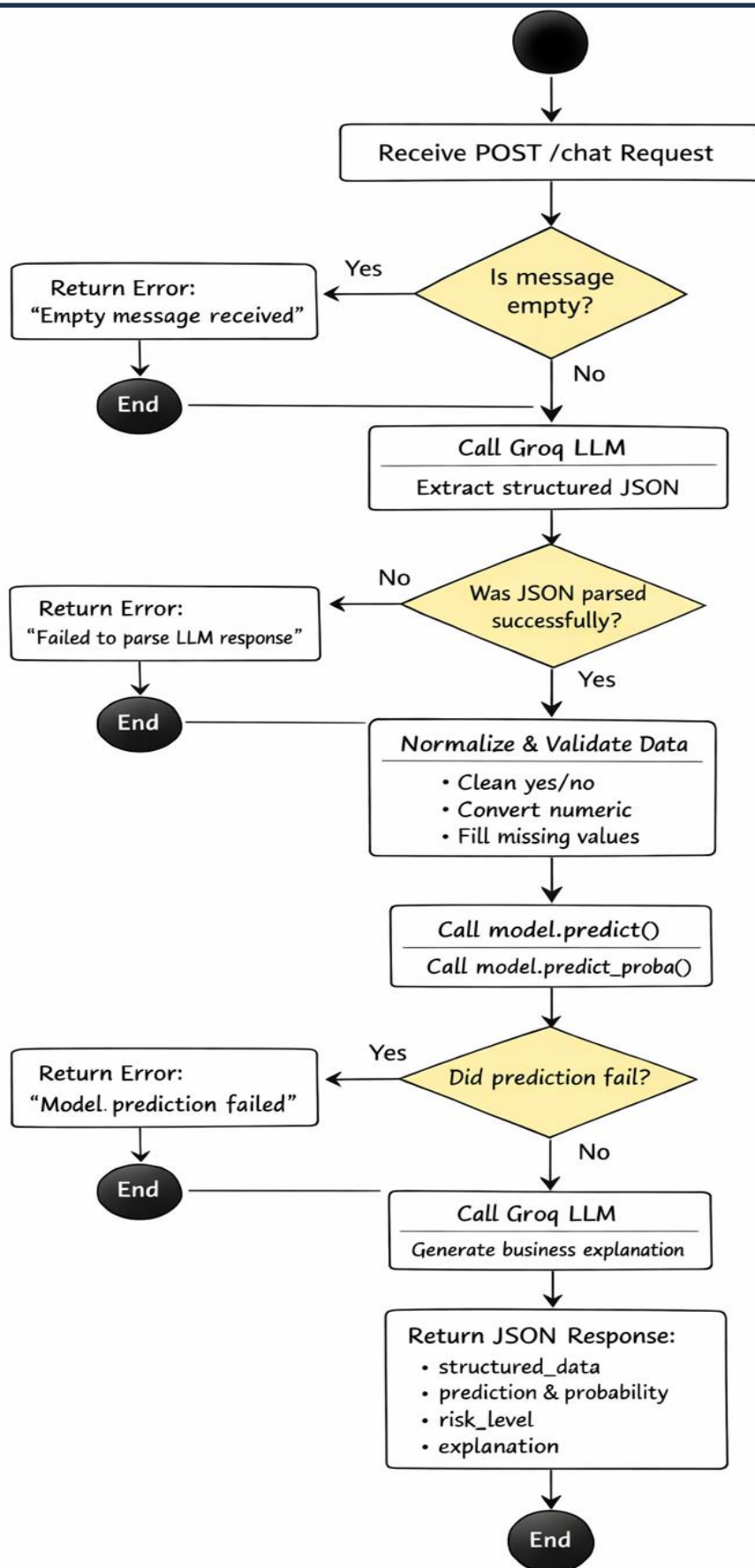
Additional handling:

- Yes/No standardization
- Numeric conversion
- Auto-calculation of Total_Charges if missing

This ensures stable inference and protects the ML model.

Chapter 7

LLM Integration Strategy



Model Used

- LLaMA 3 (Open-source)
- Accessed via Groq inference API

Why Groq Was Used

Groq was selected for:

- Rapid Proof-of-Concept development
- No GPU infrastructure requirement
- Stable low-latency responses
- Reliable structured output

The system architecture abstracts the LLM layer, allowing migration to:

- Local HuggingFace inference
- Ollama runtime
- Self-hosted deployment

There is no vendor lock-in in the design.

Chapter 8

Streamlit Frontend

The frontend provides:

- Clean UI
- Dynamic input text area
- Risk visualization (color-coded)
- Probability display
- Expandable structured JSON view
- Error messages for API failures

The UI communicates with FastAPI via HTTP POST.

Chapter 9

Risk Analysis & Mitigation

Risk 1: LLM JSON Failure

Mitigation:

- Strict prompt instructions
- Safe JSON parsing
- Controlled error response

Risk 2: Feature Mismatch

Mitigation:

- `model.feature_names_in_` alignment
- Missing column auto-fill

Risk 3: Invalid Numeric Data

Mitigation:

- Float casting
- Fallback values
- Auto `Total_Charges` computation

Risk 4: Model Prediction Failure

Mitigation:

- Wrapped inference in `try/except`
- Controlled failure response

Risk 5: Third-Party Dependency

Mitigation:

- Modular LLM service layer
- Open-source underlying model
- Replaceable inference backend

Chapter 10

Business Insights

Model reveals:

- Month-to-month contracts increase churn risk
- Higher monthly charges correlate with churn
- Lack of tech support increases risk
- Longer tenure reduces churn probability

Business applications:

- Proactive retention targeting
- Personalized discount offers
- High-risk segmentation
- Revenue protection strategies

Chapter 11

Technical Strengths

- End-to-end AI system
- Full ML pipeline saving
- Model comparison approach
- Imbalance handling
- Robust normalization layer
- Modular architecture
- Defensive programming
- Clear separation of concerns
- Business-aligned output

Chapter 12

How to Use The AI Churn Chatbot

The AI Churn Prediction Assistant supports two interaction modes:

- Natural Language Input (Business Mode)
- Structured JSON Input (Technical Mode)

This design ensures accessibility for both non-technical marketing users and technical stakeholders.

Natural Language Mode (Recommended for Marketing Teams)

Marketing users can describe customers using plain [English](#).

“The customer is a female senior citizen with a 24 months contract.

*She uses fiber optic internet, has streaming TV and movies,
no tech support, and pays via electronic check.*

Monthly charges are 95 dollars and tenure is 18 months.”

Internal Processing Flow:

1. LLM extracts structured JSON features
2. Normalization layer standardizes values
 - "24 months" → "Two year"
 - "lady" → "Female"
 - "fiber optics" → "Fiber optic"
3. Features aligned with model schema

4. ML model generates prediction
5. LLM generates business explanation

Output Provided:

- Structured feature data
- Churn probability
- Risk level (HIGH / LOW)
- Business explanation

This mode is optimized for usability and business decision-making.

Structured JSON Mode (Technical Mode)

Technical users can directly provide structured feature data.

Example JSON Input:

```
{  
  "gender": "Female",  
  "Senior_Citizen": 1,  
  "Is_Married": "Yes",  
  "Dependents": "No",  
  "tenure": 18,  
  "Phone_Service": "Yes",
```

```
"Dual": "No",  
"Internet_Service": "Fiber optic",  
"Online_Security": "No",  
"Online_Backup": "Yes",  
"Device_Protection": "No",  
"Tech_Support": "No",  
"Streaming_TV": "Yes",  
"Streaming_Movies": "Yes",  
"Contract": "Two year",  
"Paperless_Billing": "Yes",  
"Payment_Method": "Electronic check",  
"Monthly_Charges": 95,  
"Total_Charges": 1710  
}"
```

Benefits:

- No ambiguity
- Suitable for automation
- API integration ready
- Ideal for technical validation

Automatic Validation & Normalization

Regardless of input type, the backend enforces:

- Yes/No normalization
- Numeric type casting
- Missing value handling
- Schema alignment using `model.feature_names_in_`
- Safe JSON parsing

This ensures robustness and production stability.

Chapter 13

Feature Improvements

- SHAP explainability
- Model retraining automation
- Docker containerization
- Authentication & monitoring
- Full local LLM deployment
- Advanced model tuning

Chapter 14

Conclusion

This project demonstrates:

- Strong ML engineering practices
- Production-oriented backend development
- LLM integration with validation safeguards
- Business-focused AI system design
- Robust error handling and schema enforcement

The solution transforms churn prediction from a standalone classifier into an interactive AI

powered decision assistant suitable for business teams.